CIPHERCOMPLY

UI / UX Implementation Brief

A build-ready visual and interaction specification for Claude to implement in the existing project.

Design direction	Cosmos-inspired editorial minimalism + privacy-first fintech
Product	Privacy-preserving eligibility attestation using AI-assisted document extraction, deterministic policy evaluation, and zero-knowledge verification on Midnight.
Audience	Claude Code / an implementation agent working inside the existing repository
Source of truth	CipherComply Final Implementation Specification v5 (frozen) supplied by the user

The design principle

Make the product feel calm, precise, and trustworthy. Use Cosmos for its whitespace, typography, rounded geometry, and restrained composition—not for its discovery-gallery content. The core experience must communicate document processing, privacy boundaries, eligibility, proof generation, and on-chain confirmation honestly.

Implementation constraint: this PDF specifies the UI and frontend behavior. It does not certify contract security, prove the current toolchain compiles, or authorize claims that have not been validated.

1. Product and design goals

CipherComply lets a user submit evidence for an eligibility policy and obtain a narrowly scoped attestation. Sensitive source documents remain off the public ledger; only the limited state defined by the protocol is recorded on-chain. The UI must make the process understandable without pretending that zero-knowledge proofs independently validate source documents.

Primary UX goals

- Make the next action obvious: connect wallet / authenticate, choose the policy, review processing disclosures, upload evidence.
- Show the true lifecycle: idle → uploading → analyzing → rejected / review / attested → proving → submitting → confirmed / error.
- Distinguish document/policy eligibility from a finalized on-chain verification.
- Explain what is processed by the application server and AI provider, what is public on-chain, and what retention policy actually applies.
- Keep technical details available in a secondary details view; do not put hashes and raw cryptographic metadata in the main hero.

Visual personality

Trait	Implementation
Editorial minimalism	Large, confident headings; spacious sections; restrained decoration.
Privacy-first fintech	Clear status labels, explicit consent, predictable forms, accessible error states.
Web3, used sparingly	Small wallet/network indicators and abstract cryptographic motifs; no neon overload.
Trust through honesty	Only show status backed by real backend, proof-server, wallet, and chain events.

Do not use fake live statistics, simulated chain confirmations, fabricated transaction hashes, or unsupported claims such as “100% secure” and “fully anonymous.”

2. Design tokens

Use these as initial tokens. They are proposed design choices, not existing official brand tokens.

Token	Value	Use
Canvas / warm ivory	#F7F5F1	Page background; landing-page canvas.
Surface / white	#FFFFFF	Cards, forms, dialogs, upload panel.
Ink	#171916	Primary text, logo, primary CTA.
Muted text	#6D6B65	Descriptions, helper text, metadata.
Border	#E6E1DA	Subtle outlines and dividers.
Success / emerald	#237A52	Only confirmed positive states.
Warning / amber	#B7791F	Review required, pending, caution.
Error / red	#B83A3A	Rejected, failed, blocking errors.

Typography

Role	Suggested size	Rules
Marketing hero	64–80 px desktop; clamp down on mobile	Heavy sans-serif; tight line height; slight negative tracking.
Page title	36–48 px desktop	Clear, compact, strong hierarchy.
Card title	18–24 px	Semibold; short labels.
Body	14–16 px	Readable line-height; muted text for secondary content.
Metadata	12–13 px	Use sparingly; never use tiny text for privacy disclosures.
Hashes / IDs	12–14 px monospace	Wrap safely; include copy button; do not overflow cards.

Font suggestion: Inter or Geist for general UI; Geist Mono for technical identifiers. The exact font is a project choice, not a requirement.

Spacing and shape

- Use an 8 px spacing rhythm. Typical card padding: 24–32 px; page gutters: 24–48 px on desktop.
- Cards: 16–24 px radius. Inputs and primary buttons: fully rounded pills where it suits the visual hierarchy.
- Use subtle 1 px borders instead of heavy shadows. Reserve strong contrast for headings, CTAs, and important status.
- Use one icon family consistently, preferably clean outline icons. Pair every semantic icon/color with text.

3. Global navigation and page structure

Marketing header

- Left: compact CipherComply symbol and navigation links (How it works, Privacy, For protocols).
- Center: wordmark may be centered on wide desktop; do not let it overlap the left/right groups.
- Right: Log in and a dark pill CTA such as “Start verification”.
- On mobile: collapse links into a menu; keep the primary CTA easy to find.

Authenticated app shell

- Top bar: logo, network/environment label, wallet connection state, account menu.
- Desktop: optional compact left navigation for Overview, Verifications, Privacy & settings.
- Main content: constrained readable width; use generous spacing and one obvious primary action per page.
- Show the actual selected network/environment. Never hard-code a connected wallet or production status.

Required pages (initial scope)

Page	Purpose	Primary elements
Landing	Explain value and establish trust	Hero, how-it-works, privacy boundary, CTA, FAQ/footer.
Dashboard	Show current state and next step	Start verification, active status, validity, recent verifications.
Verification wizard	Complete one verification	Policy selection, disclosures, upload, processing lifecycle.
Verification details	Explain the outcome	Eligibility result, assurance tier, validity, network, tx reference.
Privacy/settings	Give user control and transparency	Processing mode, providers, retention, wallet/session controls.

Keep an operator/admin console separate from the ordinary user experience. It can be a later phase for policy management, revocation, audit events, and operations.

4. Landing page: Cosmos-inspired, product-specific

Use a spacious warm-ivory canvas and an oversized centered headline. The visual atmosphere may use sparse, floating abstract elements around the edges, but the central text and CTA must remain unobstructed.

Hero content (recommended copy)

Eyebrow: PRIVATE ELIGIBILITY VERIFICATION
Headline: Prove your eligibility.
Second line: Keep your documents private.
Supporting copy: Verify sensitive evidence off-chain. Prove eligibility on-chain with zero-knowledge verification.
Primary CTA: Start verification
Secondary link: How it works

Hero composition

- Headline centered, large, heavy sans-serif; limit to two lines at desktop sizes.
- Primary CTA: deep-ink pill with white text. Keep the button label action-oriented.
- Optional secondary action: quiet text link, not another equally prominent filled button.
- Use floating low-contrast abstract motifs: document outline, wallet, shield, network nodes, proof/grid pattern.
- Do not use realistic bank statements, government IDs, personal income values, or fake proof data as decoration.
- Decorative layer uses pointer-events:none and stays behind content. Keep text and interactive controls accessible.
- Below hero: three or four compact steps—Submit evidence, Evaluate policy, Generate proof, Confirm on-chain.

How it works section

Step	User-facing explanation
1. Submit evidence	Choose the policy and upload the requested supported document.
2. Evaluate eligibility	The extraction pipeline reads evidence; deterministic policy rules decide the result.
3. Generate proof	The client prepares the witness/proof flow required by the protocol.
4. Confirm on-chain	The app waits for real transaction finalization and indexed status.

Include a plain-language trust-boundary section: the proof verifies the authorized attestation and specified constraints; it does not independently prove that the original document is genuine or that AI extraction was correct.

5. Dashboard specification

The dashboard is an action-oriented workspace, not a crypto analytics dashboard.

Top section

- Small eyebrow: YOUR VERIFICATION WORKSPACE.
- Headline: “Verify privately.” with a supporting line such as “Prove only what matters.”
- One sentence describing the next step.
- Wallet/network status appears in the header, not as a giant metric card.

Main upload card

- Title: “Start a verification”.
- Description: “Upload the document required by your selected policy.”
- Large drop zone with file icon, “Drop your document here”, supported file types and max size sourced from actual server configuration.
- Primary button: “Choose document”.
- Show validation errors inline, with a clear next step. Never expose raw server errors.
- Before file submission, require the user to see the applicable data-processing disclosure.

Secondary cards

Card	Contents
Verification status	Not started / Processing / Needs review / Eligible off-chain / Submitting / Confirmed / Failed.
Latest result	Policy label, outcome, assurance tier, valid-until date when known.
Network	Actual configured network/environment and connection state.
Recent activity	Timestamp, state, short reference; no sensitive document content.

Do not display a positive “Verified on Midnight” state until the transaction is finalized and the indexed verifiedUntil status is confirmed. If only the policy passed, say “Eligible — awaiting on-chain confirmation.”

6. Verification wizard and lifecycle

Use a stepper/timeline with explicit labels. Each step should be driven by actual application state, not a decorative animation or timer.

State	UI behavior	Important rule
Idle	Show policy selection and upload action.	No progress indicator.
Uploading	Show file name, size, upload progress if available, cancel if safe.	Do not claim server acceptance before response.
Analyzing	Show “Analyzing document” and a calm activity indicator.	Do not imply a fixed completion time.
Rejected	Show outcome and safe reason code in plain language.	Do not sign an ineligible attestation.
Review	Amber state; explain what needs review and what user can do.	Do not imply automatic approval.
Attested / eligible	Explain that policy evaluation passed and an attestation was issued.	Not yet equivalent to on-chain confirmation.
Proving	Show proof generation in progress.	Use real proof-server/client callbacks.
Submitting	Show transaction submitted/pending and a reference if available.	Do not display confirmed.
Confirmed	Show “Verified on Midnight”, validity and transaction details.	Only after finalization + indexed status.
Error	Show generic user-safe message and correlation ID where appropriate.	Never expose secrets, document contents, or raw stack traces.

Progress visual

- Vertical stepper on mobile; horizontal stepper is acceptable on wide screens if labels remain readable.
- Completed steps: muted emerald icon + text “Complete”. Current step: high-contrast outline/fill + “In progress”. Future steps: neutral gray.
- Color is not the only status signal; use labels and icons.
- Avoid arbitrary percentage completion when progress cannot be measured.

Failure and retry

- Explain whether retry is possible and whether the existing request may already have been issued.
- Respect idempotency semantics; do not create a fresh attestation simply because the UI was refreshed.
- Show a retry action only when the backend workflow says retry is safe.

7. Privacy disclosures and trust language

Place the processing disclosure before upload/consent, not buried in the footer.

Required disclosure content

- What data is uploaded and which application service receives it.
- Whether the configured mode is server-mediated, TEE-backed, or on-device; only show modes actually implemented.
- Whether the AI provider can see plaintext under the selected mode.
- What information is public on-chain: explain that ledger fields are public and pseudonymous identifiers may remain linkable.
- Retention duration, deletion behavior, and any off-chain audit retention, based on the real policy.
- A clear acknowledgement before the first upload.

Safe copy patterns

Prefer	Avoid
“Sensitive documents are not written to the public ledger.”	“Your data never leaves your device.” (unless true for the selected mode)
“The document is processed by our server and configured AI provider.”	“Nobody can see your document.”
“Eligible — awaiting on-chain confirmation.”	“Verified” immediately after AI/policy pass.
“The proof checks the attestation and defined constraints.”	“ZK proves your document is authentic.”
“Public chain data may be linkable to a pseudonymous subject.”	“Completely anonymous.”

The interface should not overstate the guarantees of the protocol. The oracle/document pipeline remains a trust boundary, and the UI should explain it in clear language.

8. Verification details page

The details page serves users who need evidence that a result exists, without revealing the underlying document.

Field	Display behavior
Outcome	Separate eligibility decision from on-chain confirmation state.
Assurance tier	Display signed assuranceLevel (Low / Medium / High) as authoritative; verificationMethod is descriptive metadata only.
Policy	Readable policy name/version plus policy ID in a secondary details area.
Validity	Show verified-until date only when sourced from confirmed state; make expiration explicit.
Network	Actual Midnight network/environment.
Transaction reference	Shortened identifier with copy action and external explorer link only if the correct URL is configured.
Proof details	Show non-sensitive technical metadata only when useful; never expose private witness or user secret.
Document	Show safe filename/receipt metadata only if retention policy permits; never show raw extracted sensitive fields by default.

Use disclosure/accordion sections for “Technical details” and “Privacy details”. Keep the main result scannable.

9. Responsive and accessibility requirements

- Desktop: centered landing hero; app content constrained to a readable max width; dashboard cards in a two-column grid where useful.
- Tablet: reduce decorative imagery, retain comfortable card padding, allow nav to collapse if needed.
- Mobile: stack all cards, use a vertical progress stepper, make upload and CTA controls full width, keep disclosures readable.
- Never let floating decoration overlap text, forms, wallet controls, or status indicators.
- Keyboard: visible focus ring, logical tab order, keyboard-accessible file picker and dialogs.
- Accessibility: semantic headings, labeled inputs, announced async status changes, sufficient contrast, reduced-motion support.
- Use prefers-reduced-motion; animation must not be necessary to understand state.

10. Motion and component behavior

Component	Behavior
Button	Subtle hover/pressed state; clear disabled/loading state; no layout shift.
Search/input	Visible focus state; validation message near the field.
Upload zone	Subtle border/background response on drag-over; show selected filename and remove/replace option.
Floating hero objects	Slow, low-amplitude drift only if performance and reduced-motion behavior are handled.
Progress stepper	State changes tied to real lifecycle events; no fake timers.
Toast	Short confirmation for copy/success; do not use toast as the only error message.
Dialog	Trap focus appropriately, support Escape where safe, return focus to trigger.
Network/wallet chip	Reflect actual connection and selected environment; include disconnected/error states.

Keep motion calm and functional. If the static page does not look polished with animations disabled, fix the composition before adding motion.

11. Technical implementation instructions for Claude

- Inspect the existing repository before changing anything. Identify framework, routes, styling system, icon library, current auth/wallet integration, API clients, and component conventions.
- Preserve the current architecture and working functionality. Do not replace the app with a separate demo or scaffold a new project unless the repository is genuinely empty.
- Implement the design as reusable components: AppShell, Navbar, Hero, Button, Card, StatusBadge, UploadDropzone, PrivacyDisclosure, VerificationStepper, VerificationSummary, TechnicalDetails.
- Keep design tokens centralized in the existing theme/CSS variables/Tailwind config. Avoid introducing a second styling system.
- Use the project's existing wallet and Midnight.js integrations. Do not hard-code wallet addresses, hashes, timers, network values, or successful statuses.
- Wire lifecycle UI to actual API responses and client/proof/chain callbacks. If an integration is missing, expose an explicit unavailable/error state and document the missing dependency rather than faking success.
- Use the current API contract and safe error semantics. Do not log PII, document contents, secrets, private witness values, or raw provider errors.
- Check loading, empty, rejected, review, pending, confirmed, disconnected, and failure states.
- Run available lint, typecheck, and tests. Report commands run, failures, and remaining blockers honestly.

Suggested reusable component tree

```
AppShell
■■■ Navbar / WalletStatus
■■■ DashboardPage
■ ■■■ PageHeading
■ ■■■ StartVerificationCard
■ ■■■ VerificationStatusCard
■ ■■■ RecentActivity
■■■ VerificationWizard
■ ■■■ PolicySelector
■ ■■■ PrivacyDisclosure
■ ■■■ UploadDropzone
■ ■■■ VerificationStepper
```

```
■ ■■■ ResultPanel
■■■ VerificationDetails
■■■ OutcomeSummary
■■■ ValidityPanel
■■■ TechnicalDetails
```

12. Copy/paste implementation prompt for Claude

Paste the following prompt into Claude Code with this PDF available in the repository/context. Claude must inspect the actual codebase and implement, not merely return a design description.

You are implementing the CipherComply UI in the existing repository. Read this UI implementation brief and the project's frozen CipherComply v5 implementation specification before editing. First inspect the repo, identify its framework, routes, design system, existing API/wallet/Midnight integrations, and current behavior. Preserve working functionality and architecture.

Goal: implement a polished, responsive, Cosmos-inspired editorial-minimalist UI for a privacy-first eligibility verification product. Use warm ivory (#F7F5F1), white surfaces, deep ink (#171916), muted text (#6D6B65), subtle borders (#E6E1DA), and restrained semantic emerald/amber/red states. Use large clean sans-serif typography, generous whitespace, rounded cards and pill CTAs, and sparse abstract cryptographic decoration only on the marketing hero.

Implement: (1) landing page with hero, how-it-works, privacy boundary, CTA; (2) dashboard with start-verification card, actual wallet/network state, current status and recent activity; (3) verification wizard with policy selection, pre-upload privacy disclosure, upload, lifecycle stepper, outcomes; (4) verification details with eligibility, signed assurance tier, validity, network and transaction reference; (5) privacy/settings page if consistent with existing routing.

Correct lifecycle: idle → uploading → analyzing → rejected / review / attested → proving → submitting → confirmed / error. Drive states from real API, proof-server, wallet and chain callbacks. Show "Verified on Midnight" only after finalization and indexed status confirmation. Distinguish "Eligible — awaiting on-chain confirmation" from confirmed.

Security/privacy UI rules: disclose the actual processing mode and provider before upload; do not claim that server-mediated documents never leave the server or that ZK proves document authenticity. Do not invent stats, addresses, hashes, transaction results, retention promises or success states. Never expose raw document contents, PII, secrets, private witness data, or raw server errors in logs/UI. Use safe reason codes and correlation IDs when available.

Engineering rules: use existing components and integrations; do not scaffold a parallel app or swap core dependencies without a strong reason. Centralize tokens. Ensure responsive desktop/tablet/mobile layouts, keyboard accessibility, visible focus, semantic labels, reduced motion, and no decorative overlap with controls. Handle loading, empty, disconnected, review, rejected, pending, confirmed and error states.

Process: make changes in small coherent steps; run existing lint/typecheck/tests; fix issues introduced by your changes. At the end, summarize files changed, routes implemented, integrations actually wired, commands/tests run, and blockers. Do not claim the UI is production-secure or the contract is validated merely because the frontend builds.

Acceptance checklist

- The primary action is obvious on landing and dashboard.
- Every lifecycle state has a truthful, readable UI.
- The pre-upload privacy disclosure reflects the real configured mode.
- No hard-coded fake verification or chain success is possible.
- Mobile layout is usable and controls are keyboard accessible.
- The project's existing integration and architecture remain intact.
- Tests/build status and unimplemented dependencies are reported honestly.

13. Acceptance and release boundaries

This brief is a frontend design and implementation guide. It is not a replacement for the frozen protocol specification or security review.

Frontend acceptance

- Landing, dashboard, verification wizard, and details pages match the proposed design system.
- Responsive layout works at desktop, tablet, and mobile widths.
- Upload validation and error states are readable and do not leak sensitive details.
- The state machine is wired to actual events, including finalization/indexing.
- No raw PII or secrets are included in analytics, console logs, or user-facing errors.
- Loading, empty, review, rejected, pending, confirmed, disconnected, and failure states are covered.

Do not confuse UI completion with protocol readiness

The supplied v5 specification explicitly lists validation items that must be resolved against the live toolchain, including the chain-time expiry primitive, exact cryptographic call signatures, matching Profile A signing primitive, current Gemini file/MIME limits, and an independent security audit. Do not change these protocol requirements in order to make the UI appear complete. The UI may be developed, but deployment readiness must follow the specification's actual gates.

Source alignment

This brief derives product behavior from the supplied CipherComply v5 specification, particularly its product/trust model, frontend lifecycle, API contract, privacy boundary, and validation gates. Color values, typography, component layouts, and marketing copy are proposed UI decisions created for implementation and should be adjusted to fit the existing repository.

END OF IMPLEMENTATION BRIEF

Use this document as the visual and interaction contract. Use the frozen CipherComply v5 specification as the protocol and security contract.